

Firewalling and intrusion detection with Suricata

Follow-along: play the attacker, then the defender

Overview

Two mechanisms in this lab act on the same packets and do different jobs. A **packet filter** reads the source and destination addresses in a packet's IP header and the port numbers and flags in its TCP header, compares them against a list of rules, and decides whether the packet is passed to the program waiting for it or discarded. An **intrusion detection system** compares traffic against a library of signatures, each one a pattern plus a name, and writes a line naming the signature every time traffic matches. The filter decides; the detection system describes. A filter that discards a packet leaves a counter and nothing else, and a detection system reading copies of packets changes nothing about whether they arrive.

Definition: Packet filter, intrusion detection system, intrusion prevention system

A **packet filter** decides, per packet, from header fields alone, whether that packet is passed on or discarded. An **intrusion detection system (IDS)** matches traffic against named signatures and records what matched; it sits beside the traffic and does not decide its fate. An **intrusion prevention system (IPS)** is the same signature engine placed in the path, so that a signature which matches can also withhold the packet. Suricata is one program and runs as either: which one it is depends entirely on where the packets it inspects come from, and Part 3 is that change.

You will play both sides on one network. As the attacker you scan a server from outside the site, find its telnet service and log in to it. As the defender you write the server's own filter rules, watch a rule that names one outside network get walked around, and then write the rule that does not. Then you write the same policy again as Suricata signatures: first as an IDS, where every attempt is recorded and every attempt still succeeds, and then as an IPS, where the signature that matches is the thing that discards the packet.

The signatures in this lab match on addresses, port numbers and TCP flags, all of which are in the clear in every packet. Signature matching on the contents of an encrypted session, sensor placement across a network of more than one server, and detection that flags traffic for being unusual rather than for matching a pattern are all out of scope here. So is everything that is not TCP: the scan below probes TCP ports only, both signatures name a TCP port, and a service reachable over UDP would appear in none of it.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

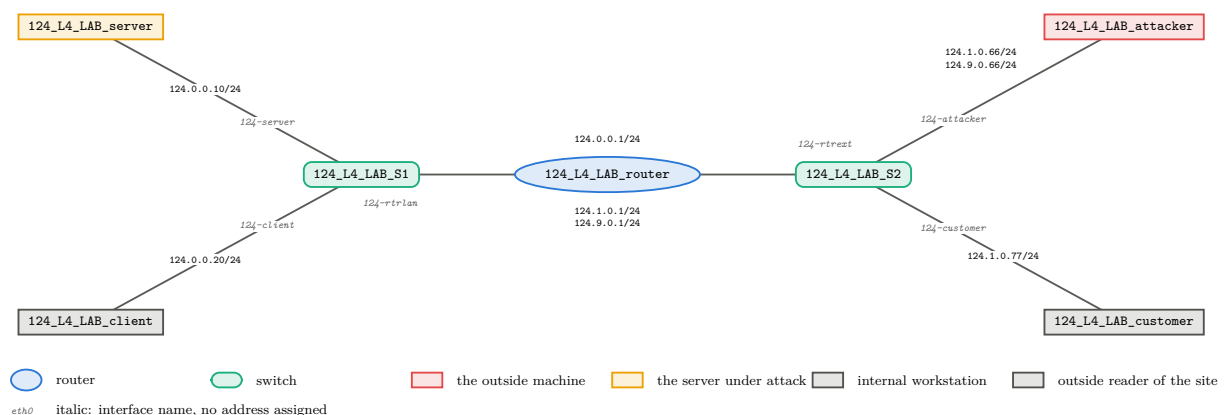
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Layer 3 and 4 attacks and mitigations**, then **Firewalling and intrusion detection with Suricata**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: the rules on the server, what the detection engine is doing, which machines currently reach the telnet service, and whether the web site still answers. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. You need two shells on the server and one on the attacker, and both server shells come from the same line.
- **Reset to baseline** throws away every rule you wrote on the server, stops the detection engine, puts its configuration back as the package shipped it, and empties the alert log, without tearing the lab down. It is the way out of a server that has stopped answering anything.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminals. **Back** (or Esc) returns to the lab list.

The setup



The site network is 124.0.0.0/24. Everything else is outside it. The outside segment carries two prefixes, 124.1.0.0/24 and 124.9.0.0/24, and the attacker's machine holds an address on each; both are ordinary addresses of that machine, and Part 1 turns on the fact that it has two.

Role	Container	Address	What it is
server	124_L4_LAB_server	124.0.0.10	the machine you defend
client	124_L4_LAB_client	124.0.0.20	a workstation inside the site
router	124_L4_LAB_router	124.0.0.1, 124.1.0.1, 124.9.0.1	forwards, filters nothing
attacker	124_L4_LAB_attacker	124.1.0.66, 124.9.0.66	the machine you attack from
customer	124_L4_LAB_customer	124.1.0.77	reads the public web site

The server's own interface is called `124-lan`. Its name is used three times in this lab: in the `tcpdump` command that captures on it, in a filter rule that matches packets arriving on it, and in the argument that tells Suricata which interface to read.

The server runs three listeners:

- `tcp/22`, `sshd`. Remote administration, over an encrypted channel.
- `tcp/23`, `telnetd`. The same job as `ssh` and none of the protection: it prompts for a username and a password and carries both, and everything after them, as plain text. The account `svcops` has the password `labpass`.
- `tcp/80`, `lighttpd`. The site's public web page, which is meant to be readable from anywhere and has to stay that way.

The attacker's machine has `nmap` and `nping` on it, and a `telnet` client. Nothing in this lab sends traffic on its own: every packet you will count is one you asked a machine to send.

How you are assessed

There are eighteen questions, placed at the step that produces the answer. Answer all eighteen in a separate document, numbered to match. Several ask you to report a number you read off your own screen, so run the command before you answer.

The terminals you need

Open three:

```
docker exec -it 124_L4_LAB_server bash      # terminal 1: the server
docker exec -it 124_L4_LAB_server bash      # terminal 2: the server again, for \
the log tail
docker exec -it 124_L4_LAB_attacker bash    # terminal 3: the attacker
```

Terminals 1 and 2 are both on the server. Terminal 2 does nothing until Part 2, when it holds a running tail of the alert log.

Part 1: the server's own packet filter

Every packet arriving at the server passes through the kernel's own filtering machinery before it reaches the program listening for it. On Linux that machinery is `netfilter`, and `iptables` is the command that writes rules into it. Nothing in the server's configuration so far uses it: the filter table is empty, so every packet that reaches the machine is passed to whatever is listening.

Definition: Table, chain, rule, target

iptables organises rules into **tables** by what the rules do. This lab uses only the **filter** table, which decides whether a packet is passed on or discarded, and which is the table **iptables** works on when no **-t** option is given. Each table holds **chains**, and a chain is a list of rules applied at one point in a packet's journey: the **INPUT** chain is applied to packets addressed to this machine, **OUTPUT** to packets this machine is sending, and **FORWARD** to packets this machine is routing on behalf of someone else. A **rule** is a set of conditions plus a **target**, named with **-j**, which says what happens to a packet that meets them. Rules in a chain are tried in order.

Look at the empty table first, so you know what a change to it looks like.

```
# 124_L4_LAB_server: the server
iptables -L
```

Three chains, no rules in any of them, and each chain's policy is **ACCEPT**, which is what happens to a packet no rule in that chain matched.

Finding the target

Definition: Port scan

A **port scan** sends one probe per port and sorts the ports by what came back. **nmap** run as root sends a TCP SYN to each port it tries: a port that replies with SYN-ACK is reported **open**, a port that replies with RST is reported **closed**, and a port that replies with nothing at all is reported **filtered**, because silence is also what a discarded probe looks like. Run without a port list, **nmap** tries its own list of 1000 ports rather than all 65535, so what it reports is a sample of the machine and not a survey of it.

Scan the server from the attacker's machine.

```
# 124_L4_LAB_attacker: the outside machine
nmap 124.0.0.10
```

Question 1. Besides **ssh** on port 22, which ports does **nmap** report open on the server, and what service name does it print against each?

One probe at a time

nmap reports on a whole machine at once. The rest of Part 1 needs a smaller instrument: one packet, built exactly as you say, so that a rule either matches it or does not.

Definition: nping

nping builds packets to order and prints one line per packet sent and one per reply received. **--tcp** selects TCP, **-p** the destination port, **--flags** the TCP flags to set, **-c** how many to send, and **--delay** how long to wait between them. Its default is five packets one second apart, which is what **-c 5 --delay 1s** below states explicitly. Lines beginning **SENT** are packets it built; lines beginning **RCVD** are replies, and the letters after the port pair are the flags each reply carried, so **SA** is a SYN-ACK.

Send five TCP SYNs, one second apart, at the server's telnet port.

```
# 124_L4_LAB_attacker
nping --tcp -p 23 --flags syn -c 5 --delay 1s 124.0.0.10
```

Question 2. Give the RCVD line for one of the five packets, and the summary line that begins **Raw packets sent**. State what the reply establishes about port 23 on the server, and say why it establishes nothing about port 25.

What the open port is worth

A port that replies to a SYN is a service that will talk to whoever is asking. This one prompts for a username and a password.

```
# 124_L4_LAB_attacker
telnet 124.0.0.10
# log in as svcops with the password labpass, then run:
id
hostname
exit
```

Question 3. You now have a shell on the server from a machine outside the site. Name which of the three listeners made that possible, and state the two header fields a filter rule would have to match on to stop it while leaving the other two listeners reachable.

Seeing the traffic

Before writing any rule, look at what arrives. `tcpdump` prints one line per packet it sees on an interface.

```
# 124_L4_LAB_server, terminal 1
tcpdump -n -i 124-lan tcp port 23
```

`-n` prints addresses as numbers instead of looking up names for them, and `-i 124-lan` restricts it to the server's own interface. Leave it running, repeat the `nping` command on the attacker, then stop `tcpdump` with Ctrl-C.

Question 4. Give one line of `tcpdump` output produced by the probe. Identify in it the source address, the destination port, and the TCP flags the packet carried.

A rule that writes something down

You now know an address that is sending connection attempts at a service that should not be replying to it. You do not yet know whether that machine is the only one, so the first rule records traffic from its whole network rather than from the one address.

Definition: The LOG target, and rule counters

LOG makes a record of the packet and then, unlike every other target in this lab, **returns to the next rule in the chain**, so a packet that a LOG rule matched carries on being tested by the rules below it. Every rule also carries two counters, one for packets and one for bytes, incremented each time that rule matches. `iptables -L --verbose` prints them, and they are how you tell a rule that is matching from a rule that is written correctly and never reached. Inside a container the kernel's log has nowhere to go, so the counters are the record you read here.

```
# 124_L4_LAB_server, terminal 1
iptables -I INPUT --source 124.1.0.0/24 -j LOG --log-prefix 'SUSPECT '
iptables -L --verbose
```

-I inserts at the top of the chain; -A would append to the bottom. Read the counters on the new rule, run the five-packet `nping` again on the attacker, and read them once more.

Question 5. Report the rule's packet counter before the probe and after it. `nping` built its five SYN's itself and handed them to a raw socket, so the attacker's kernel holds no connection for the replies. Use that to explain the relationship between the counter you read and the number five.

Question 6. Predict, before you run it. The next rule you insert goes to the top of the chain, above the LOG rule. It matches TCP packets whose source is in 124.1.0.0/24 and whose destination port is 23, and its target is DROP. Say what changes in `nping`'s output on the attacker, and what happens to the LOG rule's counter.

A rule that stops something

Definition: Matching parameters, and the DROP target

The conditions in a rule are **matching parameters**: `--source` and `--destination` take an address or a prefix, `--protocol` takes `tcp`, `udp`, `icmp` or a protocol number, and `-i` takes the name of the interface the packet arrived on. Selecting `--protocol tcp` makes the TCP-specific parameters available, of which this lab uses `--dport`, the destination port. DROP discards the packet and stops the chain: no rule below it is tried, and nothing is sent back to the sender, so the sender sees silence rather than a refusal.

```
# 124_L4_LAB_server, terminal 1
iptables -I INPUT --protocol tcp --source 124.1.0.0/24 --dport 23 -j DROP
iptables -L --verbose
```

Run the five-packet `nping` on the attacker again.

Question 7. Report `nping`'s summary line and the packet counters on both rules. Explain the relationship between the two counters from where each rule sits in the chain and what each of the two targets does when it matches.

The attacker moves

The attacker's machine holds a second address, 124.9.0.66, on the other outside prefix. -S tells `nping` which of the machine's addresses to send from.

```
# 124_L4_LAB_attacker
nping --tcp -p 23 --flags syn -c 5 --delay 1s -S 124.9.0.66 124.0.0.10
```

Question 8. Report `nping`'s summary line for this run, and the counters on the two rules afterwards. Say which rules matched these packets and which did not, and why.

The rule that names what is allowed

A rule that names the address the attacker is using is a rule the attacker rewrites by changing address. A rule that names every outside prefix has the same problem one step later: the outside is every address that is not yours, and you cannot list it.

The way out is to invert what the rule names. State the one thing you are willing to accept connections to port 23 from, and discard the rest. The `!` operator negates a matching parameter, so `! --source 124.0.0.0/24` matches every packet whose source is not in the site network. `!` is also special to bash, so it needs single quotes around it or a backslash in front of it.

Delete the two rules you have (`iptables -F` empties the chain), then write the rule. You may do it as one negated `DROP` rule or as an `ACCEPT` followed by a `DROP`; both work, and the second needs you to think about which order they go in.

Question 9. Give the rule or rules you wrote, exactly as you typed them. Then check your work: the probe from 124.1.0.66 and the probe from 124.9.0.66 should both come back with nothing, a probe from the client at 124.0.0.20 should still get replies, and `curl http://124.0.0.10/` from the customer should still return the page. Report all four results.

Question 10. Your rule names the site network. Explain what it covers that a rule naming 124.1.0.0/24 and 124.9.0.0/24 together does not, and name the one thing an attacker would have to obtain to get past it.

Keeping a record of what you discard

A `DROP` rule leaves a counter and nothing else, and a counter cannot say when the attempts came or how many machines made them. Put a `LOG` rule above it, matching the same packets, so that every attempt is recorded before it is discarded. It goes above rather than below because `DROP` ends the chain.

```
# 124_L4_LAB_server, terminal 1
iptables -I INPUT --protocol tcp --dport 23 '!' --source 124.0.0.0/24 -j LOG \
    --log-prefix 'TELNET-OUTSIDE '
iptables -L --verbose
```

Referring to a rule by its number

Rules can be deleted by repeating them in full after `-D`, or by their position in the chain.

```
# 124_L4_LAB_server, terminal 1
iptables -L --line-numbers
iptables -D INPUT 1
iptables -L --line-numbers
```

Question 11. Give the numbered list before the deletion and after it. State which rule now holds number 1, and what that means for a script that deletes several rules by number in one run.

Empty the chain before Part 2, so that nothing left over from Part 1 settles Part 2's question for it:

```
# 124_L4_LAB_server, terminal 1
iptables -F
iptables -L
```

Part 2: Suricata reading copies of the traffic

The filter you wrote in Part 1 knows one thing about the packets it discarded: how many there were. It cannot say when they arrived, what they were an attempt at, or how one attempt differed from another, because a counter is all a rule leaves behind. Suricata does the other job. It matches traffic against named signatures and writes a line for each match, carrying the time, the name you gave the signature, its identifier, a classification, and the addresses and ports of the packet that matched.

In this part Suricata reads copies of packets off the server's interface. Copies are the whole point: the original has already been delivered by the time Suricata looks at it, so nothing here can stop anything. That is the difference the last question of this part is about.

Definition: Signature, action, sid, classtype

A Suricata **signature** (also called a rule) is one line with two parts. The header names an **action**, a protocol, a source address and port, a direction, and a destination address and port. The options in brackets after it carry a **msg**, the text written into the log; a **sid**, the signature's identifier, unique across every rule file loaded; a **rev**, its revision number; and a **classtype**, a label from a shipped list that sets the priority the alert is recorded with. The actions this lab uses are **alert**, which records the match, and **drop**, which records it and discards the packet, and which does nothing at all unless Suricata is inline. Identifiers from 1000000 to 1999999 are reserved for locally written rules.

Telling Suricata about your rules file

Suricata's shipped configuration lists the rule files it loads. Yours is not one of them yet.

```
# 124_L4_LAB_server, terminal 1
nano /etc/suricata/suricata.yaml
```

Find the **rule-files:** section (Ctrl-W in **nano** searches) and add the second line, so that it reads:

```
# 124_L4_LAB_server: this is the file's content, not commands to type
rule-files:
- suricata.rules
- /etc/suricata/rules/local.rules
```

Save with Ctrl-O and leave with Ctrl-X. **suricata.rules** is the set that came with the package; yours is an addition to it, not a replacement for it. Check the edit landed:


```
# 124_L4_LAB_server, terminal 1
grep -A2 '^rule-files:' /etc/suricata/suricata.yaml
```

The first signature

Write one signature that records a connection attempt to the telnet service from inside the site.

```
# 124_L4_LAB_server, terminal 1
echo 'alert tcp 124.0.0.0/24 any -> 124.0.0.10 23 (msg:"INTERNAL Telnet connection \
      attempt from the site network"; flags:S; classtype:attempted-recon; \
      sid:1000001; rev:1;)' > /etc/suricata/rules/local.rules
cat /etc/suricata/rules/local.rules
```

The single quotes hold the whole rule together so bash leaves the double quotes inside it alone. Reading the header from the left: **alert** is the action, **tcp** the protocol, **124.0.0.0/24 any** the source address and source port to match, **->** the direction, and **124.0.0.10 23** the destination address and port.

Definition: The flags keyword

flags:S matches a TCP segment in which the SYN flag is set and no other flag is, which is the first packet of a connection and not the SYN-ACK that replies to it or any segment that follows. Suricata therefore records the attempt as the attempt arrives, rather than waiting to see whether the connection was completed. Modifiers change this: **flags:S+** would match a segment with SYN set whatever else is set with it, so it would match SYN-ACK as well.

Starting Suricata

```
# 124_L4_LAB_server, terminal 1
suricata -c /etc/suricata/suricata.yaml -i 124-lan -v > \
  /var/log/suricata/stdout.log 2>&1 &
```

Definition: Standard output, standard error, and redirection

A command writes its results to **standard output** and its diagnostics to **standard error**, two separate streams that both arrive at the terminal by default. **>** sends standard output to a file, and **2>&1** sends standard error to wherever standard output is now going, so the two together put everything the command prints into one file. The trailing **&** runs the command in the background and gives the shell back to you.

Suricata compiles every signature in every rule file before it reads its first packet, and the package's own file holds tens of thousands of them, so this takes twenty to thirty seconds. It is ready when it has printed one particular line:

```
# 124_L4_LAB_server, terminal 1: repeat until it prints a line
grep 'Engine started' /var/log/suricata/stdout.log
```

Look at the rest of that file as well. One warning about your rule is expected: Suricata reports that a SYN-only rule aimed at a port cannot match in the direction from the server back to the

client, and disables it for that direction, which is what you want. It prints one such line per signature, so there will be two of them once you add the second rule.

Watching the alerts

Definition: fast.log

Suricata writes several logs. `fast.log` is the one line per alert, in the order the alerts happened: timestamp, then `[1:sid:rev]`, then the `msg` text, then the classification and priority, then the protocol and the source and destination of the packet that matched.

In the second server terminal, start a tail that keeps printing new lines as they are written:

```
# 124_L4_LAB_server, terminal 2
tail -f /var/log/suricata/fast.log
```

Now send the five-packet probe from the client, which is inside the site:

```
# one command, run from your own terminal
docker exec 124_L4_LAB_client nping --tcp -p 23 --flags syn -c 5 --delay 1s \
    124.0.0.10
```

Question 12. Give one alert line from `fast.log`. Count the lines the probe produced, and explain that count in terms of what `flags:S` matches and how many packets the probe sent.

A second signature for everything else

One signature covers connection attempts from inside the site. A second covers the rest, and the two together are worth more than one that covers both, because the same event from inside and from outside is not the same event. The two must have different identifiers.

```
# 124_L4_LAB_server, terminal 1: >> appends, so the first rule stays
echo 'alert tcp [!124.0.0.0/24] any -> 124.0.0.10 23 (msg:"EXTERNAL Telnet \
    connection attempt from outside the site"; flags:S; classtype:attempted-recon; \
    sid:1000002; rev:1;)' >> /etc/suricata/rules/local.rules
cat /etc/suricata/rules/local.rules
```

`[!124.0.0.0/24]` is the same negation Part 1 wrote with `!`, in Suricata's own syntax: every address that is not in the site network.

Suricata reads its rule files once, at startup. Rather than stop and restart it, tell the running process to reload them:

```
# 124_L4_LAB_server, terminal 1
suricatasc -c reload-rules
```

It returns immediately, but the reload itself compiles every rule again and takes as long as the start did. Watch for the second of the two lines it writes:

```
# 124_L4_LAB_server, terminal 1: repeat until the second line appears
grep 'rule reload' /var/log/suricata/stdout.log
```

Now probe from both of the attacker's addresses:

```
# 124_L4_LAB_attacker
nping --tcp -p 23 --flags syn -c 5 --delay 1s 124.0.0.10
nping --tcp -p 23 --flags syn -c 5 --delay 1s -S 124.9.0.66 124.0.0.10
```

Question 13. Report which signature identifier fired for each of the two runs and how many lines each produced. Then say what `nping` reported for each run, and what those two results together say about the state of the server.

What the record is worth on its own

Log in over telnet from the attacker again, the same way you did in Part 1, and watch the tail in terminal 2 while you do it.

Question 14. State what `fast.log` recorded during the login and what the attacker ended up with. Suricata matched the packet and named it correctly. Explain, from where Suricata is getting its packets, why that made no difference to the outcome.

Part 3: Suricata in the path

The defence in this part changes one thing: where Suricata's packets come from. Instead of reading copies off the interface while the originals are delivered, it takes each packet out of the kernel and returns a decision on it, and the kernel does what it is told. Every signature you have already written keeps working unchanged; the one whose action you change from `alert` to `drop` now discards what it matches, and the one you leave as `alert` keeps recording without interfering.

Suricata cannot take packets off an interface itself in this mode. It needs them handed to it, and the thing that hands them over is an `iptables` rule.

Definition: Netfilter queue, and a verdict

The `NFQUEUE` target passes a packet to a numbered queue in the kernel and holds it there instead of deciding its fate. A userspace program reads packets from that queue and returns a **verdict** on each one, `ACCEPT` or `DROP`, which the kernel then applies. While a queue has no program reading it, packets sent to it are discarded, so a queue rule installed before its reader is running blocks the traffic it matches.

Handing the packets over

```
# 124_L4_LAB_server, terminal 1
iptables -I INPUT -i 124-lan -j NFQUEUE --queue-num 0
iptables -L --verbose
```

Suricata is still running in the mode Part 2 started it in, reading the interface, so nothing is reading queue 0.

Question 15. Predict what a request to the web site from the customer does now, and why. Then measure it with `docker exec 124_L4_LAB_customer curl -sS -m 8 http://124.0.0.10/` and report the line `curl` printed.

Configuring the queue reader

```
# 124_L4_LAB_server, terminal 1
nano /etc/suricata/suricata.yaml
```

Find the `nfq:` section and remove the `#` from the front of every line in it except `batchcount`, so that it reads:

```
# 124_L4_LAB_server: this is the file's content, not commands to type
nfq:
  mode: accept
  repeat-mark: 1
  repeat-mask: 1
  bypass-mark: 1
  bypass-mask: 1
  route-queue: 2
# batchcount: 20
fail-open: yes
```

`mode: accept` is the setting this lab depends on: it makes Suricata return `ACCEPT` for every packet no signature told it to discard. `fail-open: yes` makes it accept packets rather than hold them when its queue fills, which keeps a slow Suricata from becoming an outage. `batchcount` stays commented out because Suricata only reads it in its `workers` runmode, and the default runmode for a queue is `autofp`; uncommenting it produces an error line at startup and no other effect.

Changing one action

Rewrite the rules file with the outside signature's action changed and the inside one left alone:

```
# 124_L4_LAB_server, terminal 1
echo 'alert tcp 124.0.0.0/24 any -> 124.0.0.10 23 (msg:"INTERNAL Telnet connection \
  attempt from the site network"; flags:S; classtype:attempted-recon; \
  sid:1000001; rev:1;)' > /etc/suricata/rules/local.rules
echo 'drop tcp [!124.0.0.0/24] any -> 124.0.0.10 23 (msg:"EXTERNAL Telnet \
  connection attempt from outside the site"; flags:S; classtype:attempted-recon; \
  sid:1000002; rev:1;)' >> /etc/suricata/rules/local.rules
cat /etc/suricata/rules/local.rules
```

Stop Suricata and start it again taking packets from queue 0. `-q 0` replaces `-i 124-lan`: they are alternative sources of packets, not two things Suricata does at once.

```
# 124_L4_LAB_server, terminal 1
pkill -x suricata
sleep 3
suricata -c /etc/suricata/suricata.yaml -v -q 0 > /var/log/suricata/stdout.log 2>&1 &
```

Wait for `Engine started` in the log again, the same way as before. The customer's web request works again as soon as it appears, which is worth checking before you go on.

Measuring the difference

Probe from both attacker addresses, then try the telnet login again, watching the tail in terminal 2:

```
# 124_L4_LAB_attacker
nping --tcp -p 23 --flags syn -c 5 --delay 1s 124.0.0.10
nping --tcp -p 23 --flags syn -c 5 --delay 1s -S 124.9.0.66 124.0.0.10
# this one hangs and prints nothing: with the SYN discarded, nothing ever
# replies, and the kernel retries for over two minutes before it gives up.
# Press Ctrl-C after a few seconds.
telnet 124.0.0.10
```

Question 16. Report `nping`'s summary line for one of the two runs and what the `telnet` client did. Give one `fast.log` line from these probes and name what it carries that the lines in Part 2 did not.

Checking the rest of the site still works

A rule that stops the attacker and also stops everyone else has not defended anything. Three things have to still work:

```
# one command each, run from your own terminal
docker exec 124_L4_LAB_client nping --tcp -p 23 --flags syn -c 3 --delay 1s \
    124.0.0.10
docker exec -it 124_L4_LAB_client telnet 124.0.0.10
docker exec 124_L4_LAB_customer curl -s -o /dev/null -w '%{http_code}\n' \
    http://124.0.0.10/
```

Question 17. Report what each of the three produced, and what `fast.log` recorded for the client's telnet login. Name the part of the `drop` signature that let the client through, and the part that let the customer's web request through.

Question 18. The Part 1 rule and the Part 3 signature discard the same packets. Name one thing the `iptables` rule does that the signature does not, and one thing the signature does that the `iptables` rule does not.

Checks: what "done" means

Run the Status action, or `scripts/status.sh`. It prints the server's filter table, what Suricata is doing and which rules it has loaded, and then sends probes from each machine in the lab and reports what came back. The four marks at the foot of its output are the lab's oracle, and the first three are the ones that matter together:

- neither of the attacker's addresses reaches `tcp/23`;
- the workstation on the inside still does;
- the web site still replies to the customer on the outside;
- Suricata has a record of what arrived from outside.

Any one of the first three is easy on its own. A server that replies to nobody has the first. A server with no rules at all has the other two.

If the server has stopped replying to anything and you cannot see why, the likely cause is an **NFQUEUE** rule in the chain with nothing reading the queue. Status says so when it finds that state. Reset to baseline clears it.

Safety note

Every container in this lab sits on two isolated Open vSwitch bridges with no route off the host. The scanning and probing tools here are aimed only at addresses this lab created, and nothing in it can reach anything else on your machine or the network it is attached to. Running the same tools against a machine you have not been given permission to test is a different act, and in most jurisdictions an offence.

Tear the lab down

When you have answered the eighteen questions, remove the lab: pick **Teardown** in the TUI, or run **scripts/teardown.sh**. It removes this lab's seven containers and nothing else.